

Clasificación supervisada y semisupervisada

LDA, EDDA y mixturas gaussianas semisupervisadas con iris

Resumen

En esta parte seguimos trabajando con la idea de **clasificar observaciones**, pero ahora la lógica cambia.

En clustering, los grupos se construían sin conocer una etiqueta real. El método miraba la estructura de los datos y proponía grupos.

En clasificación supervisada, en cambio, los grupos ya están definidos. El modelo recibe ejemplos con su clase real y aprende una regla para clasificar nuevas observaciones.

En este apunte vamos a trabajar con `iris`, porque es un dataset chico y permite ver muy bien la idea.

Vamos a ver tres enfoques:

1. **LDA**: busca ejes lineales que separen grupos conocidos.
2. **EDDA**: usa mixturas gaussianas y permite formas de grupo más flexibles.
3. **Clasificación semisupervisada**: usa solo una parte de las etiquetas conocidas y completa el resto usando la estructura de los datos.

La idea no es memorizar funciones, sino entender qué pregunta responde cada método, qué código mínimo necesitamos y cómo leer las salidas principales.

Preparación

```
library(MASS)
library(mclust)
```

Package 'mclust' version 6.1.2

Type 'citation("mclust")' for citing this R package in publications.

```
data(iris)

# Variables predictoras
X_iris <- iris[, 1:4]

# Grupo conocido
y_iris <- iris$Species
```

```
y_iris <- iris$species
```

```
str(X_iris)
```

```
'data.frame':  150 obs. of  4 variables:
 $ Sepal.Length: num  5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
 $ Sepal.Width : num  3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1 ...
 $ Petal.Length: num  1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5 ...
 $ Petal.Width : num  0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1 ...
```

```
table(y_iris)
```

```
y_iris
      setosa versicolor  virginica
      50         50         50
```

`iris` tiene cuatro variables numéricas y una variable de grupo: `Species`.

Las cuatro variables numéricas serán las variables que usamos para clasificar. `Species` será la clase real conocida.

LDA

Idea principal

LDA significa **Linear Discriminant Analysis**, o análisis discriminante lineal.

La pregunta que responde es:

usando las variables numéricas, ¿puedo separar las especies conocidas?

LDA construye nuevas variables llamadas **funciones discriminantes**. Estas son combinaciones lineales de las variables originales.

Como `iris` tiene tres especies, LDA puede construir como máximo dos ejes discriminantes:

número de ejes = cantidad de grupos - 1
número de ejes = 3 - 1 = 2

Por eso aparecen `LD1` y `LD2`.

Ajuste del modelo

```
lda_iris <- lda(Species ~ ., data = iris)
```

```
lda_iris
```

Call:

```
lda(Species ~ ., data = iris)
```

Prior probabilities of groups:

```
      setosa versicolor virginica  
0.3333333  0.3333333  0.3333333
```

Group means:

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
setosa	5.006	3.428	1.462	0.246
versicolor	5.936	2.770	4.260	1.326
virginica	6.588	2.974	5.552	2.026

Coefficients of linear discriminants:

	LD1	LD2
Sepal.Length	0.8293776	-0.02410215
Sepal.Width	1.5344731	-2.16452123
Petal.Length	-2.2012117	0.93192121
Petal.Width	-2.8104603	-2.83918785

Proportion of trace:

LD1	LD2
0.9912	0.0088

La salida tiene tres partes importantes:

- **Prior probabilities of groups:** proporción inicial de cada especie.
- **Group means:** medias de cada variable dentro de cada especie.
- **Coefficients of linear discriminants:** pesos de cada variable en [LD1](#) y [LD2](#).

En [iris](#), las especies están balanceadas: hay 50 flores de cada una. Por eso las probabilidades previas son iguales.

Las medias por grupo muestran que [setosa](#) tiene pétalos mucho más pequeños, mientras que [versicolor](#) y [virginica](#) son más parecidas entre sí.

Predicción sobre los mismos datos

Primero usamos el modelo para predecir las mismas observaciones con las que fue entrenado.

Esto sirve para entender el método, pero no sirve como evaluación real, porque el modelo ya vio esos datos.

```
pred_lda <- predict(lda_iris)

names(pred_lda)
```

```
[1] "class"      "posterior" "x"
```

`predict(lda_iris)` devuelve tres cosas:

- `class`: clase predicha para cada observación.
- `posterior`: probabilidad estimada de pertenecer a cada especie.
- `x`: coordenadas de cada observación en los ejes discriminantes.

```
# Clase predicha
head(pred_lda$class)
```

```
[1] setosa setosa setosa setosa setosa setosa
Levels: setosa versicolor virginica
```

```
# Probabilidades posteriores
head(pred_lda$posterior)
```

	setosa	versicolor	virginica
1	1	3.896358e-22	2.611168e-42
2	1	7.217970e-18	5.042143e-37
3	1	1.463849e-19	4.675932e-39
4	1	1.268536e-16	3.566610e-35
5	1	1.637387e-22	1.082605e-42
6	1	3.883282e-21	4.566540e-40

```
# Coordenadas en LD1 y LD2
head(pred_lda$x)
```

	LD1	LD2
1	8.061800	-0.3004206
2	7.128688	0.7866604
3	7.489828	0.2653845
4	6.813201	0.6706311
5	8.132309	-0.5144625
6	7.701947	-1.4617210

Cada fila de `posterior` suma 1. La clase predicha es la que tiene mayor probabilidad.

Matriz de confusión aparente

```
tabla_lda_aparente <- table(real = y_iris,  
                             predicho = pred_lda$class)
```

```
tabla_lda_aparente
```

	predicho		
real	setosa	versicolor	virginica
setosa	50	0	0
versicolor	0	48	2
virginica	0	1	49

```
error_lda_aparente <- mean(pred_lda$class != y_iris)  
error_lda_aparente
```

```
[1] 0.02
```

La matriz compara la especie real con la especie predicha.

La diagonal son los aciertos. Los valores fuera de la diagonal son errores.

En este caso, LDA clasifica perfectamente a **setosa** y comete pocos errores entre **versicolor** y **virginica**.

El error es bajo, pero se llama **error aparente** porque se calcula sobre los mismos datos usados para entrenar.

Gráfico de los ejes discriminantes

Además de clasificar, LDA también puede verse como una forma de reducción de dimensionalidad supervisada.

PCA busca ejes que expliquen variación total. LDA busca ejes que separen grupos conocidos.

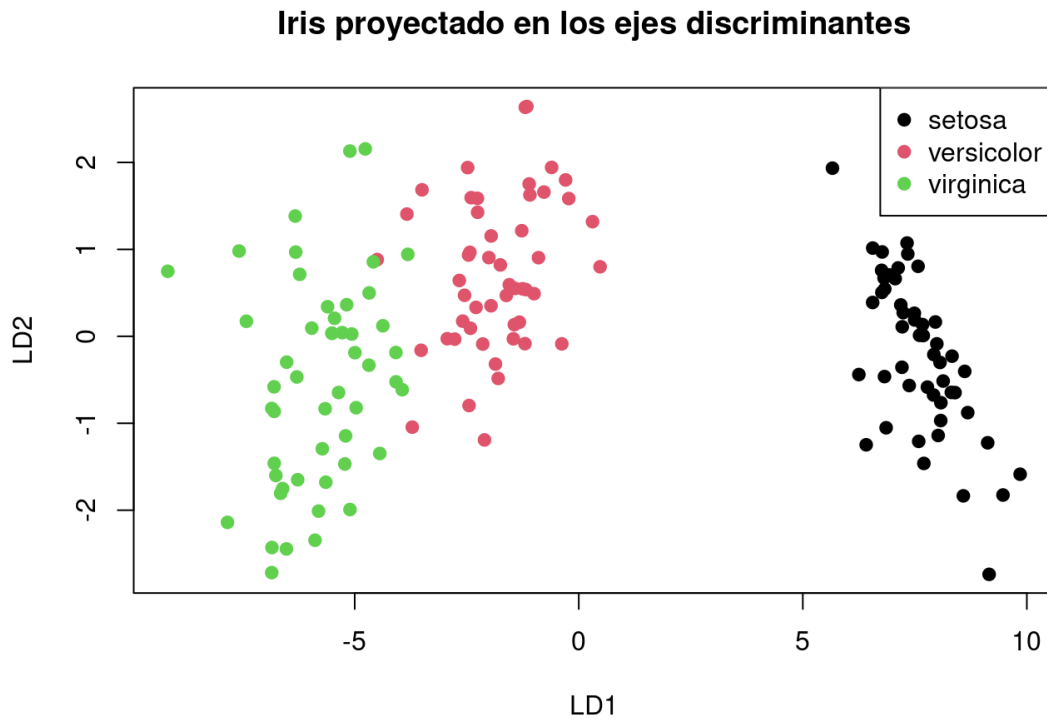
```
scores_lda <- as.data.frame(pred_lda$x)  
scores_lda$Species <- y_iris  
  
plot(scores_lda$LD1, scores_lda$LD2,  
      col = as.numeric(scores_lda$Species),  
      pch = 19,  
      xlab = "LD1",  
      ylab = "LD2",
```

```

main = "Iris proyectado en los ejes discriminantes")

legend("topright",
      legend = levels(scores_lda$Species),
      col = 1:3,
      pch = 19)

```



En el gráfico, cada punto es una flor y el color indica la especie real.

LD1 separa muy bien a *setosa* del resto. LD2 ayuda a separar *versicolor* y *virginica*, aunque esas dos especies quedan más cerca.

Validación de LDA

Hasta ahora evaluamos el modelo con los mismos datos con los que fue entrenado. Eso suele dar resultados optimistas.

Ahora vamos a ver dos formas más honestas de evaluar.

Leave-one-out

Con `CV = TRUE`, `lda()` aplica validación leave-one-out.

La idea es:

saco una flor, entreno con las otras 149 y predigo la flor que dejé afuera.

Eso se repite para todas las flores.

```
lda_iris_cv <- lda(Species ~ ., data = iris, CV = TRUE)

tabla_lda_cv <- table(real = y_iris,
                      predicho = lda_iris_cv$class)

tabla_lda_cv
```

	predicho		
real	setosa	versicolor	virginica
setosa	50	0	0
versicolor	0	48	2
virginica	0	1	49

```
error_lda_cv <- mean(lda_iris_cv$class != y_iris)
error_lda_cv
```

```
[1] 0.02
```

Esta evaluación es menos optimista que el error aparente, porque cada flor se predice con un modelo que no la usó para entrenar.

En este ejemplo, el error sigue siendo bajo. Eso indica que la separación entre especies es fuerte.

Entrenamiento y prueba

Otra forma común es dividir los datos en entrenamiento y prueba.

Usamos el 70% para entrenar y el 30% para evaluar.

```
set.seed(123)

n <- nrow(iris)

ind_train <- sample(1:n, size = round(0.7 * n))
ind_test  <- setdiff(1:n, ind_train)

iris_train <- iris[ind_train, ]
iris_test  <- iris[ind_test, ]
```

```
table(iris_train$Species)
```

setosa	versicolor	virginica
36	32	37

```
table(iris_test$Species)
```

setosa	versicolor	virginica
14	18	13

La partición no queda perfectamente balanceada, pero las tres especies están representadas en entrenamiento y prueba.

```
lda_iris_train <- lda(Species ~ ., data = iris_train)
```

```
lda_iris_train
```

Call:

```
lda(Species ~ ., data = iris_train)
```

Prior probabilities of groups:

setosa	versicolor	virginica
0.3428571	0.3047619	0.3523810

Group means:

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width
setosa	4.966667	3.394444	1.461111	0.2555556
versicolor	5.971875	2.787500	4.309375	1.3500000
virginica	6.586486	2.948649	5.529730	1.9945946

Coefficients of linear discriminants:

	LD1	LD2
Sepal.Length	0.887242	-0.1977697
Sepal.Width	1.267866	-1.8705232
Petal.Length	-2.101235	1.1717820
Petal.Width	-2.934756	-3.1681626

Proportion of trace:

LD1	LD2
0.9937	0.0063

Ahora predecimos las flores de prueba.

```
pred_lda_test <- predict(lda_iris_train,  
                          newdata = iris_test)
```



```
tabla_lda_test <- table(real = iris_test$Species,  
                        predicho = pred_lda_test$class)
```

```
tabla_lda_test
```

	predicho		
real	setosa	versicolor	virginica
setosa	14	0	0
versicolor	0	17	1
virginica	0	0	13

```
error_lda_test <- mean(pred_lda_test$class != iris_test$Species)  
error_lda_test
```

```
[1] 0.02222222
```

Esta evaluación se parece más a una situación real, porque las flores de prueba no fueron usadas para ajustar el modelo.

EDDA

Idea principal

EDDA es otro clasificador supervisado.

También usa grupos conocidos, pero trabaja con **mixturas gaussianas**.

Una forma simple de pensarlo:

cada especie se representa como una nube gaussiana multivariada.

EDDA permite que esas nubes tengan estructuras de covarianza más flexibles que LDA. Por eso puede adaptarse mejor cuando los grupos tienen formas distintas.

La función que usamos es `MclustDA()` del paquete `mclust`.

Ajuste del modelo EDDA

```
edda_iris <- MclustDA(X_iris,  
                     class = y_iris,  
                     modelType = "EDDA")
```

```
summary(edda_iris)
```

```
-----  
Gaussian finite mixture model for classification  
-----
```

EDDA

```
-----  
log-likelihood   n df      BIC  
      -187.7097 150 38 -565.8236
```

Classes	n	%	Model	G
setosa	50	33.33	VEV	1
versicolor	50	33.33	VEV	1
virginica	50	33.33	VEV	1

Training confusion matrix

```
-----  
              Predicted  
Classes      setosa versicolor virginica  
setosa         50           0           0  
versicolor     0           47           3  
virginica       0           0          50
```

Classification error = 0.0200

Brier score = 0.0127

En el resumen miramos:

- cantidad de observaciones por especie,
- modelo de covarianza seleccionado,
- matriz de confusión de entrenamiento,
- error de clasificación.

G = 1 indica que se usó una gaussiana por especie.

El nombre del modelo, por ejemplo **VEV**, describe la estructura de covarianza elegida. Para este apunte alcanza con saber que esa estructura fue seleccionada por BIC.

Predicción aparente con EDDA

```
pred_edda <- predict(edda_iris, newdata = X_iris)
```

```
assign_edda <- pred_edda$classification
```

```
tabla_edda_aparente <- table(real = y_iris,
                             predicho = asign_edda)
```

```
tabla_edda_aparente
```

	predicho		
real	setosa	versicolor	virginica
setosa	50	0	0
versicolor	0	47	3
virginica	0	0	50

```
error_edda_aparente <- mean(asign_edda != y_iris)
error_edda_aparente
```

```
[1] 0.02
```

Al igual que antes, esta es una evaluación aparente, porque se predice sobre los datos usados para ajustar el modelo.

En *iris*, EDDA también clasifica muy bien. Los errores suelen aparecer entre *versicolor* y *virginica*.

EDDA con entrenamiento y prueba

Usamos la misma partición 70/30 que para LDA.

```
edda_iris_train <- MclustDA(iris_train[, 1:4],
                           class = iris_train$Species,
                           modelType = "EDDA")

summary(edda_iris_train)
```

```
-----
Gaussian finite mixture model for classification
-----
```

```
EDDA
```

```
-----
log-likelihood   n df      BIC
-146.5773 105 38 -470.0051
```

Classes	n	% Model	G
setosa	36	34.29	VEV 1
versicolor	32	30.48	VEV 1
virginica	37	35.24	VEV 1

Training confusion matrix

Classes	Predicted		
	setosa	versicolor	virginica
setosa	36	0	0
versicolor	0	30	2
virginica	0	0	37

Classification error = 0.0190

Brier score = 0.0105

Ahora predecimos sobre `iris_test`.

```
pred_edda_test <- predict(edda_iris_train,
                          newdata = iris_test[, 1:4])

tabla_edda_test <- table(real = iris_test$Species,
                         predicho = pred_edda_test$classification)

tabla_edda_test
```

real	predicho		
	setosa	versicolor	virginica
setosa	14	0	0
versicolor	0	17	1
virginica	0	0	13

```
error_edda_test <- mean(pred_edda_test$classification != iris_test$Species)
error_edda_test
```

[1] 0.02222222

En este ejemplo, EDDA y LDA suelen tener rendimientos muy parecidos.

Eso no significa que EDDA no sirva. Significa que `iris` ya está muy bien separado y una frontera lineal alcanza para clasificar casi todo.

Clasificación semisupervisada con mixturas gaussianas

Idea principal

En LDA y EDDA conocemos todas las etiquetas del entrenamiento.

Gaussian finite mixture model for semi-supervised classification

```
log-likelihood    n df          BIC
      -187.4946 150 38    -565.3933
```

```
Classes          n      % Model G
  setosa          14   9.33   VEV 1
 versicolor      18  12.00   VEV 1
 virginica       18  12.00   VEV 1
  <NA>          100  66.67
```

Classification summary

```
-----
              Predicted
Classes      setosa versicolor virginica
  setosa         14           0          0
versicolor        0          18          0
 virginica        0           0         18
  <NA>          36          29         35
```

En el resumen, la fila **<NA>** es la más interesante. Muestra cómo el modelo clasificó las observaciones cuya especie fue ocultada.

Evaluar las etiquetas ocultas

Como nosotros ocultamos las etiquetas artificialmente, todavía conocemos la especie real en **y_iris**.

Eso permite comparar predicción contra realidad.

```
pred_semi <- predict(clasi_semi,
                     newdata = X_iris[is.na(y_semi), ])

tabla_semi <- table(real = y_iris[is.na(y_semi)],
                   predicho = pred_semi$classification)

tabla_semi
```

```
              predicho
real          setosa versicolor virginica
  setosa         36           0          0
versicolor        0          29          3
 virginica        0           0         32
```

```
error_semi <- mean(pred_semi$classification != y_iris[is.na(y_semi)])
error_semi
```

```
[1] 0.03
```

Esta evaluación no es igual a un train/test clásico.

El modelo vio las variables de todas las flores, pero no vio las etiquetas de las observaciones marcadas como **NA**.

Por eso la pregunta acá es:

si conozco solo algunas etiquetas, ¿puedo recuperar las etiquetas faltantes usando la estructura de los datos?

Comparación final

```
comparacion <- data.frame(
  Metodo = c("LDA aparente", "LDA leave-one-out", "LDA test",
             "EDDA aparente", "EDDA test", "Semi supervisado"),
  Error = c(error_lda_aparente, error_lda_cv, error_lda_test,
            error_edda_aparente, error_edda_test, error_semi)
)

comparacion
```

	Metodo	Error
1	LDA aparente	0.02000000
2	LDA leave-one-out	0.02000000
3	LDA test	0.02222222
4	EDDA aparente	0.02000000
5	EDDA test	0.02222222
6	Semi supervisado	0.03000000

En este ejemplo, todos los métodos funcionan bien porque **iris** tiene especies bastante separables.

setosa casi siempre queda perfectamente separada. La mayor confusión aparece entre **versicolor** y **virginica**.

La idea importante no es que un método “gane” siempre, sino entender qué lógica usa cada uno.

Cierre

LDA, EDDA y la clasificación semisupervisada trabajan con la misma idea general: asignar observaciones a clases.

La diferencia está en cuánta información usan y cómo representan los grupos.

LDA usa una regla lineal. EDDA usa nubes gaussianas más flexibles. El modelo semisupervisado usa solo una parte de las etiquetas y aprovecha la estructura del conjunto completo.

En *iris*, los tres enfoques funcionan bien porque las especies están bastante separadas por las variables medidas.